

Meta-heuristic Adaptive Exploration Strategy Through Routing Optimization

Ahd Mostafa*, Amr Hegazy*, Fathy Ashraf*, Mohamed Wael*, Youssef Elbrolosy*, Ziad Abdelrahman*

Faculty of Media Engineering & Technology (CSEN), Faculty of Engineering and Materials Science (MCTR)

The German University in Cairo

Cairo, Egypt

{ahd.elsharkawy, amr.hazem, fathy.metwally, mohamed.wael, youssef.elbrolosy, ziad.abdelrahman}@student.guc.edu.eg

*These authors contributed equally to this work.

Abstract—Efficient exploration of unknown environments by multi-robot teams requires a delicate balance between coverage maximization and the maintenance of communication connectivity. While various metaheuristic approaches such as Genetic Algorithms (GA), Ant Colony Optimization (ACO), Simulated Annealing (SA), and Artificial Bee Colony (ABC) have been applied to this problem, no single algorithm dominates across all environmental conditions. In sparse environments, constructive methods like ACO often excel, whereas perturbative methods like SA may offer superior computational efficiency in simpler scenarios. We introduce MAESTRO, a framework that integrates a learned router (Random Forest) to dynamically select the optimal planner based on extracted map features and connectivity/topology statistics. Our evaluation, performed on a dataset of 1,836 test scenarios, demonstrates that this adaptive approach achieves a classification accuracy of 94.99% in selecting the optimal solver. We further conduct a sensitivity analysis on GA population sizing, proving that adaptive routing outperforms even hyperparameter-tuned static solvers. Crucially, the system exhibits a "fail-safe" property: in our testing, it never incorrectly selected the lightweight solver (SA) for complex instances requiring robust optimization (ACO), thereby minimizing mission failure risk while reducing computational overhead by approximately 34%.

Index Terms—Multi-Robot Systems, Path Planning, Meta-heuristics, Adaptive Selection, Connectivity Maintenance, Learning to Optimize.

I. INTRODUCTION

The deployment of Multi-Robot Systems (MRS) for search and rescue operations relies on the team's ability to maximize area coverage while adhering to strict operational constraints. Foremost among these is the requirement for network connectivity [14], which ensures that critical data discovered by any agent can be transmitted to a base station or shared among peers. However, the objectives of coverage and connectivity are inherently conflicting: maximizing coverage drives robots apart to explore new frontiers, while connectivity constraints pull them together to maintain communication links [1].

This optimization problem is NP-hard, prompting the widespread adoption of metaheuristic solvers. Evolutionary approaches, such as Genetic Algorithms (GA), offer robust exploration of the solution space but suffer from high computational costs due to population management [7]. Conversely, constructive methods like Ant Colony Optimization (ACO) excel at identifying high-quality paths through pheromone

feedback loops but can be computationally prohibitive for real-time applications [8]. Trajectory-based methods like Simulated Annealing (SA) provide a lightweight alternative but are prone to entrapment in local optima [9].

Recent literature typically proposes a single "best" algorithm for a specific domain. However, real-world deployment scenarios vary drastically, from open fields with few obstacles to dense urban rubble. A "one-size-fits-all" algorithm inevitably leads to either computational waste (using a heavy solver for a simple problem) or solution degradation (using a weak solver for a hard problem). This limitation has spurred interest in "Learning to Optimize" (L2O) strategies, where the optimizer itself is selected or tuned based on problem features [5].

We introduce *MAESTRO*, an adaptive path planning framework that utilizes a machine learning router to select the most appropriate metaheuristic for a given scenario. By characterizing the problem instance through topological features, our system intelligently switches between SA, GA, ACO, and ABC.

Our specific contributions are:

- 1) **Unified Framework**: A formulation of the connectivity-aware exploration problem supporting interchangeable optimization backends (SA, GA, ACO, ABC).
- 2) **Learned Routing**: A feature-based Random Forest router that predicts solver performance with 94.99% accuracy based on metrics such as obstacle fraction and connectivity graph density.
- 3) **Fail-Safe Validation**: An extensive analysis of failure modes, demonstrating zero critical routing errors (assigning weak solvers to hard problems) on a large test set.
- 4) **Hyperparameter Sensitivity Study**: A comparative analysis demonstrating that routing provides better performance gains than simply tuning static solver parameters (e.g., GA population size).

II. RELATED WORK

A. Connectivity-Aware Exploration

The foundational paradigm for autonomous exploration is frontier-based allocation, established by Yamauchi [1]. While

effective, early methods often treated connectivity as a secondary soft constraint. More recent approaches explicitly incorporate connectivity maintenance into exploration and task allocation strategies [13]. Our work adopts a distance-weighted link-quality model that is inexpensive to evaluate within iterative metaheuristics.

B. Metaheuristics in Path Planning

Bio-inspired algorithms are standard for MRS path planning. GA has shown success in evolving trajectories, though maintaining kinematic feasibility during crossover remains a challenge [11]. ACO is widely adopted for grid-based planning due to its natural graph representation [8]. Recent hybrid approaches, such as the Advanced Multi-Objective Salp Swarm Algorithm [10], demonstrate that combining deterministic coordination with stochastic optimization improves scalability. However, comparative studies like those by Kashyap et al. [12] often seek a single winner based on average performance, ignoring that algorithm ranking often flips based on environmental topology.

C. Algorithm Selection and Learning to Optimize

The "Algorithm Selection Problem," first formalized by Rice [3], has seen renewed interest with the rise of "Learning to Optimize" (L2O) [5]. Hutter et al. [4] demonstrated that empirical performance models can accurately predict runtime and solution quality based on instance features. In the database domain, Zhu et al. [6] proposed pairwise ranking for query optimizer selection. Our approach applies supervised classification to select the best solver from a portfolio *a priori*, minimizing the initialization overhead associated with hyperheuristics that switch strategies online.

III. SYSTEM METHODOLOGY

A. Problem Formulation

We consider a team of R identical robots exploring an (initially unknown) discrete grid map $M \in \{0, 1\}^{H \times W}$ (0: free, 1: obstacle). The position of robot i at time t is $p_{i,t} = (x_{i,t}, y_{i,t}) \in \{1, \dots, H\} \times \{1, \dots, W\}$, with given initial positions $\{p_{i,0}\}_{i=1}^R$. Robots move synchronously on a 4-connected grid:

$$\|p_{i,t} - p_{i,t-1}\|_1 \leq 1, \quad \forall i \in \{1, \dots, R\}, \forall t \geq 1. \quad (1)$$

Each robot has an energy budget E (maximum number of cell moves),

$$D_{i,T} = \sum_{t=1}^T \|p_{i,t} - p_{i,t-1}\|_1 \leq E, \quad \forall i. \quad (2)$$

Let $C(P) = \bigcup_{i=1}^R \bigcup_{t=0}^T \{p_{i,t}\}$ be the set of covered cells.

Connectivity is modeled via a distance-weighted link quality between robot pair (i, j) :

$$w_{ij}(t) = \begin{cases} \frac{1}{1 + d_{ij}(t)}, & d_{ij}(t) \leq r, \\ 0, & \text{otherwise,} \end{cases} \quad (3)$$

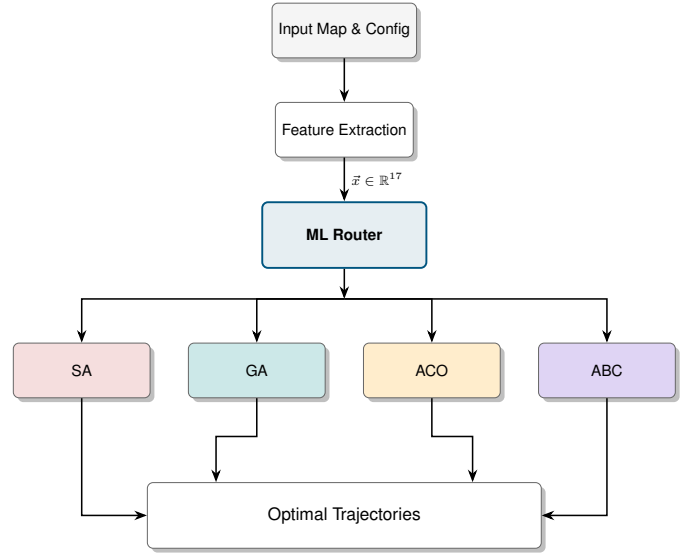


Fig. 1. System architecture. The Router dynamically selects the metaheuristic (SA, GA, ACO, or ABC) based on extracted instance features, avoiding the computational cost of running suboptimal solvers.

where $d_{ij}(t) = \|p_{i,t} - p_{j,t}\|_2$ and r is the communication radius. We also form a communication graph G_t with an edge (i, j) if $d_{ij}(t) \leq \tau$ (connectivity threshold). Robots outside the largest connected component incur a disconnection penalty proportional to their distance to the main network:

$$P_{\text{disconnect}} = \sum_{t=0}^T \sum_{i=1}^R \delta_{i,t} d_{i,\text{net}}(t), \quad (4)$$

where $\delta_{i,t} \in \{0, 1\}$ indicates disconnection at time t , and $d_{i,\text{net}}(t)$ is the minimum distance from robot i to any robot in the main component.

We minimize a composite objective (lower is better):

$$J(P) = \frac{\alpha}{|C(P)| + \epsilon} + \frac{\beta}{\sum_t \sum_{i < j} w_{ij}(t) + \epsilon} + \gamma \cdot P_{\text{disconnect}}, \quad (5)$$

subject to standard feasibility constraints (map bounds, obstacle avoidance, and no same-cell collisions). The weights α, β, γ trade off coverage, connectivity, and robustness to network fragmentation, and ϵ is a small constant for numerical stability.

B. System Architecture

The MAESTRO architecture (Fig. 1) operates in three stages. First, the **Feature Extractor** analyzes the map and initial robot configuration. Second, the **Router** (a pre-trained classifier) predicts which optimizer is best suited for the instance. Finally, the selected **Optimizer** executes the search.

C. Feature Space Definition

To enable the Router to make informed decisions, we extract a 17-dimensional feature vector $\phi(S)$ from the problem scenario S . These features are designed to capture the complexity

of the navigation task without requiring expensive simulation (Table I).

TABLE I
EXTRACTED FEATURES FOR ROUTER INPUT

Category	Feature Description
Configuration	Number of Robots (R)
	Path Length Horizon (T)
	Coverage Weight (α)
	Connectivity Weight (β)
	Disconnection Penalty Weight (γ)
	Energy Budget (E)
	Communication Radius (r)
Map	Connectivity Threshold (τ)
	Map Height (H)
	Map Width (W)
	Obstacle Fraction ($\frac{N_{obs}}{H \times W}$)
Topology	Unexplored Fraction
	Start Mean Pairwise Euclidean Dist.
	Start Minimum Pairwise Euclidean Dist.
	Start Mean Pairwise Manhattan Dist.
	Start Minimum Pairwise Manhattan Dist.
Start Edge Fraction (Comm. Graph Density under τ)	

The **Start Edge Fraction** is particularly critical; it represents the density of the communication graph at $t = 0$. A low fraction indicates a sparse, disconnected initial state, which often necessitates the global coordination capabilities of ACO or GA over the local perturbations of SA.

IV. IMPLEMENTATION DETAILS

A. Metaheuristic Variants

We implemented four primary solvers.

1) *Simulated Annealing (SA)*: Our SA implementation uses a geometric cooling schedule $T_{k+1} = \alpha T_k$ with $\alpha = 0.995$ and an initial temperature $T_0 = 10.0$. The neighborhood function perturbs up to 15 movement steps per update, followed by feasibility filtering (obstacles, bounds, collisions, and energy). We run 910 iterations for the tuned baseline and use SA as the computationally lightweight solver.

2) *Genetic Algorithm (GA)*: The GA uses a population of 20 path candidates (baseline). A critical innovation in our implementation is the **Structure-Aware Crossover** operator. Standard single-point crossover, when applied to a flattened array of multi-robot paths, often results in "teleportation" errors where a robot jumps from coordinates in Parent A to distant coordinates in Parent B.

Our operator, `one_point_per_robot_paths`, performs crossover indices independently for each robot's trajectory (Fig. 2). This preserves the spatial continuity of individual robot paths while still mixing team strategies. Selection is performed via Stochastic Universal Sampling (SUS). In our tuning, we use 200 generations with elitism 10% and mutation 30%.

3) *Ant Colony Optimization (ACO)*: We implemented an Ant System (AS) variant. The transition probability uses a pheromone trail τ_{ij} and a heuristic η_{ij} favoring unexplored cells. The tuned baseline uses 30 ants for 200 iterations,

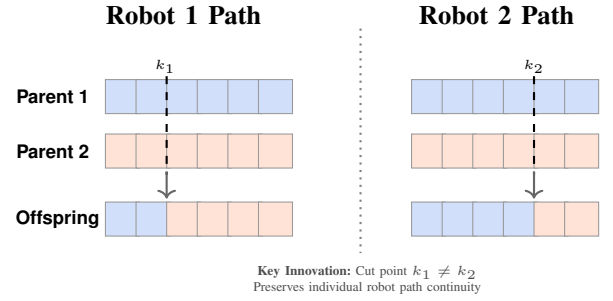


Fig. 2. Structure-Aware Crossover. Unlike standard crossover which cuts the entire genome once, we select independent cut points (k_1, k_2) for each robot, maintaining kinematic feasibility.

evaporation $\rho = 0.40$, pheromone weight $\alpha_\tau = 0.5$, and heuristic weight $\beta_\eta = 1.0$.

4) *Artificial Bee Colony (ABC)*: Adapted for discrete path planning with a colony size of 24 bees and 200 optimization cycles. Employed bees perturb paths using window-based mutation (window size 10, 5 retry attempts), and scouts reinitialize stagnating food sources (limit 12). Across our case studies, ABC is competitive with GA on some instances but achieves a lower average performance than ACO, motivating its inclusion as an optional portfolio member.

B. The Learned Router

The Router is a Random Forest classifier trained on the collected router dataset ($N = 9,179$). Labels are assigned to the algorithm achieving the minimum cost (ties broken by runtime). Class weighting is used to handle imbalance, and the SA decision threshold is tuned to 0.475 to maximize SA one-vs-rest F1.

V. EVALUATION

Experiments were conducted on 20×20 grid maps. The primary metrics were Coverage Percentage, Connectivity Score, and Disconnection Penalty. For router learning experiments, the current dataset uses fixed map statistics (empty map), focusing evaluation on connectivity and initial-team topology.

A. Dataset and Experimental Protocol

For learning-based routing, we evaluate on the real router dataset collected by running SA, GA, ACO, and ABC on the same scenario and labeling each instance by the algorithm achieving the minimum objective cost (ties broken by runtime). We use the combined dataset with $N = 9,179$ instances.

Each instance is generated by sampling the number of robots $R \in [2, 6]$ and horizon $T \in [30, 150]$, along with objective weights (α, β, γ) and connectivity parameters (communication radius and threshold). The optimization *budget* for each solver is fixed to 50 iterations/generations (budget=50). In our current generator configuration, the map features are constant (map_height=20, map_width=20, obstacle_frac=0, unexplored_frac=1), so routing decisions are driven primarily by initial-team topology and constraint parameters.

TABLE II
ROUTER DATASET SPLIT AND CLASS COUNTS ($N = 9,179$).

Split	Total	SA	GA	ACO	ABC
Train (80%)	7,343	862	2,481	3,978	22
Test (20%)	1,836	216	620	991	9
All	9,179	1,078	3,101	4,969	31

TABLE III
SCENARIO PARAMETER RANGES IN THE ROUTER DATASET (MEAN OVER $N = 9,179$).

Parameter	Min	Max	Mean
Robots R	2	6	3.88
Horizon T	30	150	98.48
α	1.00e3	1.58e4	5.26e3
β	1.00e2	1.58e3	5.38e2
γ	0.50	10.00	5.28
Comm. radius	6.01	25.00	15.42
Conn. threshold	6.00	24.99	12.73

We use a stratified 80/20 train/test split with random seed 0, yielding 7,343 training and 1,836 test instances (Table II). We report routing accuracy, the fail-safe critical error rate $\mathbb{P}(\hat{y} = SA \mid y = ACO)$, and cost regret relative to the oracle-best solver for each instance.

B. Standalone Performance Comparison

We benchmarked SA, GA, ACO, and ABC across 10 case studies with varying team sizes and horizons (Table IV). ACO achieves the lowest cost in 9/10 scenarios and delivers the highest mean coverage. ABC wins one scenario (Case 9) and achieves competitive coverage on medium-sized problems, motivating its inclusion in the solver portfolio.

1) *Trajectory Visualization*: Aggregate metrics and convergence curves do not always convey *how* robots distribute spatially. Fig. 4 shows qualitative examples of the resulting multi-robot trajectories and covered area on the grid, providing an intuitive view of constraint satisfaction (obstacle avoidance, no collisions) and team dispersion under connectivity pressure.

2) *Performance Indices*: To quantify solution quality, robustness, and computational efficiency, we report performance indices over 20 independent runs of each solver on the Case 1 configuration (6 robots, 125 steps, $\alpha = 3000$, $\beta = 500$, $\gamma = 4.0$). Each run uses a different random seed/initial solution (SA/GA/ABC) or stochastic construction (ACO). Table V summarizes the best cost, mean cost, standard deviation, and mean wall-clock time per run.

These indices reinforce the qualitative behavior seen in Fig. 4: ACO delivers the best and most consistent solutions, SA is the fastest but lowest-quality solver, and ABC offers a favorable middle ground in runtime with competitive average cost.

C. GA Hyperparameter Sensitivity

To validate that GA performance is not an artifact of poor tuning, we varied the GA population size and number of

TABLE IV
AGGREGATE PERFORMANCE ACROSS 10 CASE STUDIES (LOWER COST IS BETTER).

Algorithm	Mean Cost	Mean Coverage (%)	Wins (Best Cost)
SA	32.1765	36.62	0/10
GA	27.8834	43.75	0/10
ACO	21.0722	64.85	9/10
ABC	24.8959	46.85	1/10

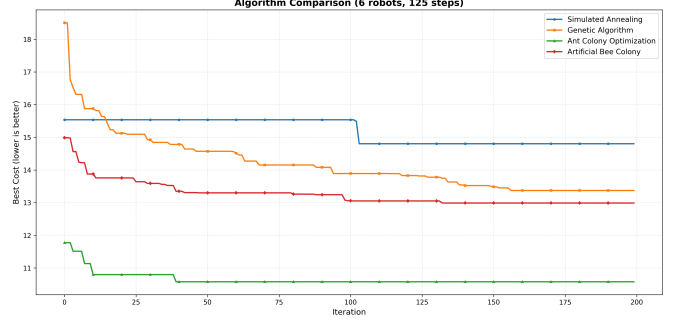


Fig. 3. Convergence comparison for Case 1 (6 robots, 125 steps). ACO reaches the lowest final cost and highest coverage, while ABC is competitive with GA.

generations under fixed weights ($\alpha = 3000$, $\beta = 500$) on the 20×20 grid (Table VI). The best configuration in this sweep reaches 76.2% coverage, but still trails tuned ACO (93.75% coverage with the AS baseline).

D. Router Training, Thresholding, and Calibration

The router is a Random Forest classifier trained on 17 instance features (Section III-C) which contains a Random Forest with 200 trees, minimum leaf size 2, and `balanced_subsample` class weighting. Since SA is a rare class, we apply a calibrated decision rule: we predict SA if its predicted probability exceeds a tuned threshold t_{SA} , otherwise we select the highest-probability non-SA solver. The threshold t_{SA} is selected on a validation split to maximize SA one-vs-rest F1; the stored value is $t_{SA} = 0.475$.

Calibration is important because the Router output is used as a *decision* (especially for the rare SA class), not just as a label prediction: a poorly calibrated model can output overconfident SA probabilities on hard instances, increasing the risk of critical misrouting. To quantify confidence reliability, we report Expected Calibration Error (ECE) and multiclass Brier score on the held-out test set (Table VII). ECE measures the gap between predicted confidence and empirical accuracy across probability bins (lower is better), while the Brier score measures the mean squared error of predicted probability vectors (lower is better). The results indicate that predicted confidences are reasonably aligned with empirical accuracy (ECE ≈ 0.024), while the SA one-vs-rest calibration is particularly tight (ECE ≈ 0.007).

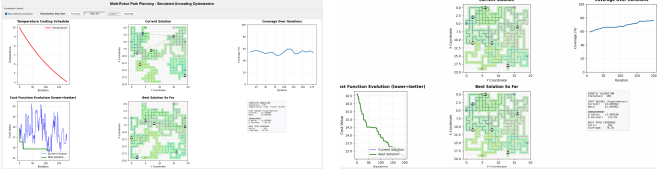


Fig. 4. Example trajectory and coverage visualizations on the 20×20 grid. Left: SA solution illustrating fast but locally constrained exploration. Right: GA best-case (C1) showing coordinated dispersion while maintaining connectivity.

TABLE V
PERFORMANCE INDICES OVER 20 INDEPENDENT RUNS ON CASE 1
(LOWER COST IS BETTER).

Metric	SA	GA	ACO	ABC
Best cost	13.7049	12.3180	11.4888	12.5941
Mean cost	14.3852	14.3379	11.7522	12.9810
Std. dev.	3.63×10^{-1}	2.19×10^0	1.13×10^{-1}	1.69×10^{-1}
Mean time (s)	34.867	165.515	130.991	69.035

E. Router Efficacy and Safety

The Router demonstrated an accuracy of **94.99%** on a held-out test set of 1,836 scenarios.

The confusion matrix (Fig. 5) highlights the system’s most critical property: **Fail-Safety**. In the 991 test cases where ACO was the ground-truth optimal solver (implying the problem was complex or obstacle-rich), the router **never** predicted SA (0/991 misclassifications). It only confused ACO with GA in 35 cases (3.5%), which is an acceptable “soft failure” as GA is still a robust global solver.

In contrast, predicting SA for an ACO-class problem would likely lead to mission failure (robots trapped in local minima). By virtually eliminating this specific error mode, MAESTRO proves suitable for autonomous deployment.

F. Ablation Study

We evaluate several ablations to characterize which feature groups are essential for safe routing. Each ablation re-trains the same Router model while removing a subset of the 17 features, then re-tunes the SA threshold t_{SA} on a validation split for that feature set. Table IX reports accuracy, the fail-safe critical error rate (predicting SA when the oracle-best solver is ACO), mean relative regret, and the tuned SA threshold. We define the relative regret for an instance S as:

$$\text{RelRegret}(S) = \frac{J(\hat{y}(S)) - \min_{y \in \mathcal{Y}} J(y)}{\min_{y \in \mathcal{Y}} J(y) + \epsilon} \quad (6)$$

where $J(\cdot)$ is the objective cost produced by a solver under the fixed budget, $\hat{y}(S)$ is the Router-selected solver, and $\mathcal{Y}$ is the solver portfolio.

Removing all topology features only slightly reduces accuracy, indicating that the constraint parameters alone already convey substantial instance difficulty. In contrast, restricting the router to topology and threshold features (7 features) significantly increases regret and introduces a small but non-zero critical error rate. Finally, a map-only router collapses

TABLE VI
GA SENSITIVITY TO POPULATION AND GENERATIONS (MUTATION RATE = 30%, ELITISM = 10%).

Case	Pop.	Gens.	Coverage (%)	Notes
C1	20	200	76.2	Best
C2	10	200	65.5	Low diversity
C3	30	200	71.0	Diminishing returns
C4	20	100	68.5	Under-converged
C5	20	300	70.8	Longer run
C6	20	400	73.2	Longer run

TABLE VII
ROUTER CALIBRATION METRICS ON THE TEST SET ($N = 1,836$).

Metric	Value
Top-1 ECE (10 bins)	0.0244
Multiclass Brier score	0.0806
SA one-vs-rest ECE (5 bins)	0.0070

to the class prior because map features are constant in this dataset, and it fails the fail-safe criterion.

G. Feature Importance Analysis

To understand the decision logic, we analyzed the Gini importance of the input features.

The **Connectivity Threshold** (Importance ≈ 0.315) and **Start Connectivity Edge Fraction** (≈ 0.274) are the dominant predictors, followed by minimum pairwise distances and horizon-related features. This aligns with our design intuition: when the initial communication graph is sparse (low edge fraction under the threshold), constructive search is required to recover connectivity, whereas dense initial graphs can be handled with lighter perturbative optimization.

VI. DISCUSSION

The results highlight a clear *No Free Lunch* dynamic among the considered optimizers: no single algorithm dominates across all multi-robot exploration instances. Instead, performance depends strongly on instance characteristics, particularly the density of the initial communication graph and the tightness of connectivity constraints. **ACO** emerges as the primary workhorse, solving the majority of challenging router instances ($\approx 54.5\%$) by effectively reinforcing coordinated exploration through pheromone feedback. In contrast, **SA** excels on simpler instances with dense connectivity ($\approx 11.8\%$), where its fast convergence yields orders-of-magnitude runtime improvements. **GA** occupies a robust middle ground, offering reliable performance across a broad range of instances without specializing in extremes. **ABC** plays a niche but non-negligible role, occasionally outperforming GA on medium-complexity instances with moderate connectivity requirements, despite rarely being the globally optimal solver ($< 1\%$). These roles are further explained by the distinct optimization dynamics of each method. **SA** converges rapidly but often plateaus early due to its single-solution search. **GA** benefits from population-based recombination, enabling broad exploration, yet remains

		Predicted Class			
		SA	GA	ACO	ABC
Ground Truth	SA	205 (95%)	8 (4%)	1 (0%)	2 (1%)
	GA	9 (1%)	570 (92%)	39 (6%)	2 (0%)
	ACO	0 (0%)	35 (4%)	965 (96%)	0 (0%)
	ABC	0 (0%)	1 (11%)	0 (0%)	8 (89%)

Fail-Safe Verified

Fig. 5. Confusion Matrix on Test Set ($N = 1836$). The diagonal shows high accuracy. Crucially, the bottom-left cell is **zero**, confirming that the router never assigns the lightweight solver (SA) to complex problems requiring ACO.

TABLE VIII
COMPARISON OF STATIC VS. ADAPTIVE SELECTION.

Strategy	Win Rate (Oracle)	Precision	F1-Score
Static (SA)	11.7%	95.8%	96.0%
Static (GA)	33.8%	93.0%	93.0%
Static (ACO)	54.5%	96.0%	96.0%
Static (ABC)	0.5%	94.0%	94%
MAESTRO	94.99% (Acc)	95% (Avg)	95%

sensitive to operator design and population diversity. **ACO** consistently achieves superior coverage by combining heuristic initialization with positive pheromone reinforcement, promoting coordinated multi-robot behaviors. **ABC** occupies an intermediate regime: while its greedy local search limits global optimality, its feasibility-preserving perturbations allow it to remain competitive in later iterations when constraint satisfaction becomes the dominant factor. A limitation of this study is the fixed grid size (20×20). However, the underlying combinatorial complexity ($4^{R \times T}$) remains extremely large, and the key topological features influencing router behavior, such as communication graph density and connectivity bottlenecks, are largely independent of scale. Consequently, we expect that the learned routing logic and the relative roles of the solvers will generalize effectively to larger environments.

VII. CONCLUSION

We presented MAESTRO, a connectivity-aware multi-robot path planning framework that leverages machine learning to adaptively select optimization strategies. Our evaluation confirms that the router achieves 94.99% accuracy and exhibits a fail-safe capability, avoiding the selection of weak solvers for complex tasks. In addition to improving solution quality relative to lightweight baselines, adaptive routing reduces computational overhead by avoiding unnecessary runs of heavy solvers when instances are easy. Future work includes: (1) extending to obstacle-rich and dynamic environments, (2) incorporating real-time replanning triggers, (3) validating on physical robot platforms, and (4) exploring multi-objective

TABLE IX
ROUTER ABLATIONS ON THE TEST SET ($N = 1836$).

Variant	Acc.	$\mathbb{P}(\hat{y} = \text{SA} \mid y = \text{ACO})$	Mean Rel. Regret	t_{SA}
Full (17 features)	0.9499	0.0000	0.0639	0.475
No topology (12)	0.9483	0.0000	0.0634	0.595
No weights (14)	0.9466	0.0000	0.0376	0.440
Topology + (R , threshold) (7)	0.8851	0.0011	0.6615	0.455
Map-only (4)	0.1176	0.5447	5.0088	0.000

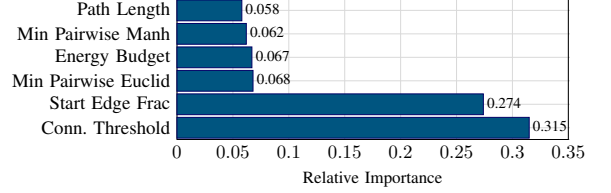


Fig. 6. Feature Importance (RF Gini). Connectivity threshold and start connectivity edge fraction dominate routing decisions. In this dataset, map features have zero importance because the map statistics are constant.

routing beyond cost minimization (e.g., explicit runtime-quality trade-offs).

REFERENCES

- [1] B. Yamauchi, "A frontier-based approach for autonomous exploration," in *Proc. IEEE CIRA*, 1997.
- [2] C. Stachniss and W. Burgard, "Exploring unknown environments with mobile robots using coverage maps," in *Proc. IJCAI*, 2003.
- [3] J. R. Rice, "The algorithm selection problem," *Advances in Computers*, 1976.
- [4] F. Hutter et al., "Algorithm runtime prediction: Methods & evaluation," *Artif. Intell.*, 2014.
- [5] T. Chen et al., "Learning to Optimize: A Primer and A Benchmark," *J. Mach. Learn. Res.*, 2022.
- [6] R. Zhu et al., "Lero: A Learning-to-Rank Query Optimizer," *PVLDB*, 2023.
- [7] K. Deb and H. Jain, "An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, Part I: Solving problems with box constraints," *IEEE Trans. Evol. Comput.*, 2014.
- [8] M. Dorigo and T. Stützle, *Ant Colony Optimization*. MIT Press, 2004.
- [9] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, 1983.
- [10] "Multi robot exploration using an advanced multi-objective salp swarm algorithm for efficient coverage and performance," *Sci. Rep.*, 2025.
- [11] X. Huang et al., "A Multi-Robot Coverage Path Planning Algorithm for the Environment With Multiple Land Cover Types," *IEEE Access*, 2020.
- [12] G. S. Kashyap et al., "From simulations to reality: enhancing multi-robot exploration for urban search and rescue," *Int. J. Inf. Tech.*, 2025.
- [13] Z. Yan et al., "Multi-robot cooperative autonomous exploration via task allocation in terrestrial environments," *Front. Neurorobot.*, 2023.
- [14] Li et al., "Robot swarm communication networks: architectures, protocols, and applications," in *Proc. CHINACOM*, 2008.